

Reinforcement Learning - Final Course Project

Maor Hornstein (ID. 300278520)

Submitted as final project report for the RL course, BIU, 2024

1 Introduction

This project presents the use of Reinforcement Learning to solve the Sokoban game, a complex puzzle involving the movement of boxes to target locations. We use Deep Q-Learning (DQN) for this purpose. This report discusses game design within the context of DQN, analyzes the impact of various hyperparameters on the achieved solutions, and presents successful experiments results.

2 Solution

2.1 General approach

The Sokoban puzzle presents a grid-based world comprising an agent, walls, boxes, and target locations. The objective is to relocate all boxes to their designated target spots while minimizing the number of actions the agent performs. The challenge lies in making optimal choices while avoiding dead-ends and loops of futile actions.

DQN represents a model-free reinforcement learning algorithm that operates on reward: this algorithm uses a neural network to approximate the Q-function, which estimates the anticipated cumulative future rewards associated with each possible action within a specific state (Q-values). By instructing the network to approximate these rewards, DQN can establish a policy that maximizes the expected rewards.

2.2 Network Architecture

We employ a deep neural network as the function approximating the Q-values. The network architecture is illustrated in figure 1 and consists of the following components:

- Input Layer: The input layer represents the current state of the Sokoban puzzle, which is encoded as an RGB picture of the game state.
- Convolution Layers: Employed for feature extraction.

- Hidden Layer: A single hidden layer with rectified linear unit (ReLU) activation function.
- Output Layer: The output layer corresponds to the Q-values for each of the 13 possible actions. It uses a linear activation function.

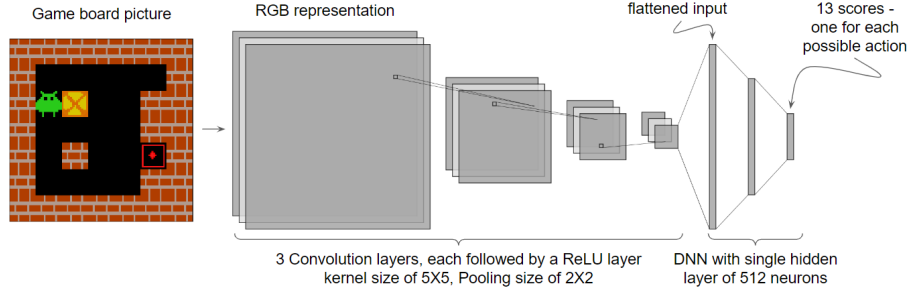


Figure 1: Network architecture illustration

2.3 The Agent Reward

We found that the most crucial aspect for the algorithm’s success lies in precisely defining the agent’s rewards.

By default, the agent loses 0.1 points for each step it takes, intended to prevent it from taking too long to solve the game. The agent earns 10 points for finishing the game, with an additional 1 point for each box placed on the target (it is penalized with -1 point if it removes the box again).

The main issue was that the agent often failed to finish the game, thereby missing the opportunity to “discover” the 10-points reward at the end. To address this, we modified the reward: we subtract the Manhattan Distance between the boxes and the targets from the predefined default reward at each step. This means the agent loses more points if the boxes are far from their targets. This adjustment encouraged the agent to strive harder to move the boxes closer to the targets, thereby increasing its chances of earning the 10 points for completing the game. It also enabled the creation of a scaled reward system, as opposed to a binary one, where the agent would either succeed and receive a reward of 10 or fail and get no reward.

This choice proved to be critical: only in this manner was it possible to guide the agent toward progressing in completing the game and to create and adapt its policy accordingly.

3 Experimental results

To examine the impact of the different hyperparameters, we methodically adjusted each hyperparameter at a time, maintaining all others at a constant level,

within a fixed scenario featuring a single-box environment. We ensured that the agent successfully completed its task in this initial configuration. Table 1 provides a summary of these constant values.

Hyperparameter	Value
Learning Rate	0.0001
Experience Replay Buffer Size	10000
Batch Size (sampled from the experience replay buffer)	32
Target Network Update Frequency (in episodes)	10
Discount Factor (Gamma)	0.999
Epsilon	1
Epsilon decay	0.999

Table 1: Summary of Constant Hyperparameter Values

For each hyperparameter tested values, we assessed 3 aspects as follows:

- **Policy Evaluation Results:** The effectiveness of the created policy was summarized in a table, indicating:
 - Whether the agent achieved its goal under the policy.
 - The number of steps taken to achieve the goal, with a maximum of 500 steps if the goal was not met.
- **Training Metrics:** The training phase evaluation is displayed in a bar chart, detailing:
 - The proportion of completed episodes out of 100.
 - The average step count per episode.
- **Training Graphs:** The training phase progress is displayed in a line graph, detailing for each of the 100 training episodes:
 - The reward obtained by the agent.
 - The number of steps taken.
 - The network loss.

We expected these graphs to show convergence, indicating increasing rewards and decreasing steps and loss.

This structured approach allowed for a clear and systematic evaluation of the impact of each hyperparameter on the performance of the DQN algorithm.

Next, we will present the above metrics for each tested hyperparameter value and discuss the results.

3.0.1 Learning Rate

The learning rate controls how quickly a model updates its knowledge, with a high rate causing fast but potentially unstable learning, and a low rate leading to steadier but slower convergence.



Figure 2: Performance Comparison of Various Learning Rate Values

It is easy to see that among the values that were tested, 0.0001 and 0.00005 are the only ones that lead to successful policy learning and therefore the only ones that are relevant to us. Of the two, 0.0001 is more successful because it leads to more episodes that end successfully during training and accordingly - in a lower number of steps.

3.0.2 Experience Replay Buffer Size

Experience replay means storing the agent's interactions, which include the state, action, reward, and next state, in a dedicated buffer. During the training process, mini-batches of experiences are randomly chosen from this buffer. This reduces the correlation between consecutive experiences during training.

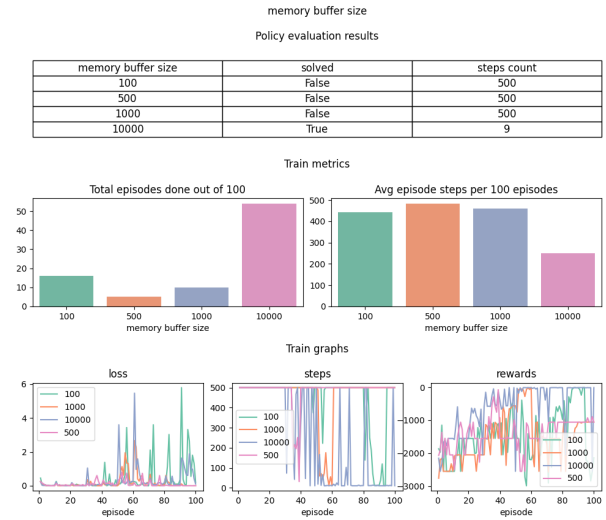


Figure 3: Performance Comparison of Various Learning Rate Values

It can be seen that in order to maximize the agent's success, it is necessary to record as much of the information it has accumulated in its trials as possible - both older and newer information. The best value, which results in the most successful episode completions and the fewest steps, is the largest one tested: 10,000.

3.0.3 Batch Size (sampled from the experience replay buffer)

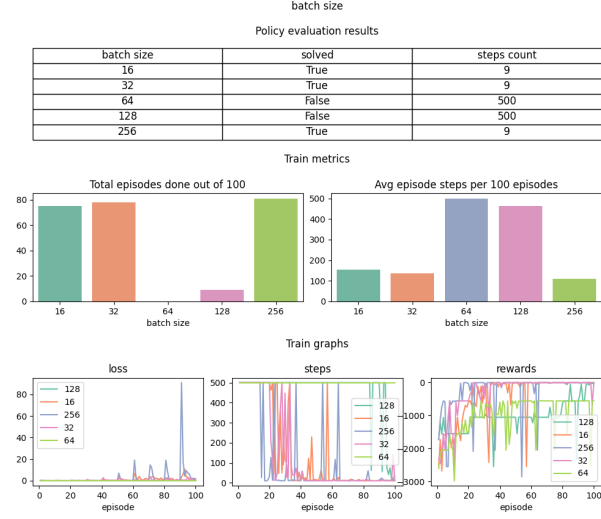


Figure 4: Performance Comparison of Various Learning Rate Values

It can be seen that the batch size that leads to the best results is 16, 32, or 256, with a slight preference for 256.

3.0.4 Target Network Update Frequency (in episodes)

The target network is a frozen snapshot of the Q-network, and we maintain it this way for a specific number of epochs each time. This prevents the issue of the target Q-values changing too much during training and improves the stability and convergence of the algorithm.

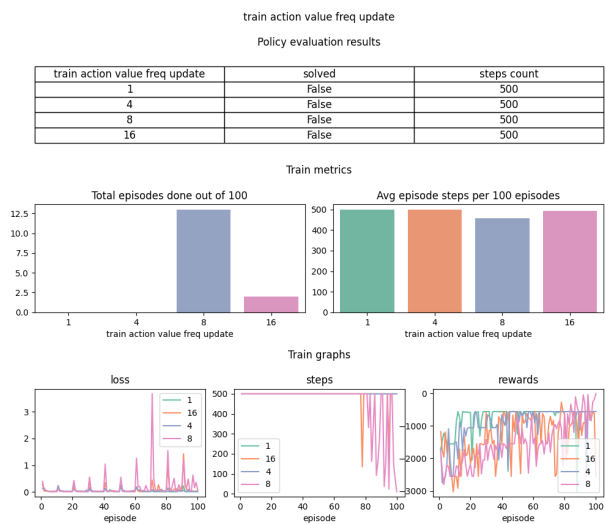


Figure 5: Performance Comparison of Various Learning Rate Values

In this case, all values led to poor results. The only value that shows a significant advantage over the others is 8, which is the closest to the default value we used of 10.

3.0.5 Discount Factor (Gamma)

The Discount Factor (Gamma) determines the importance of future rewards, with a higher value favoring long-term gains over immediate rewards.

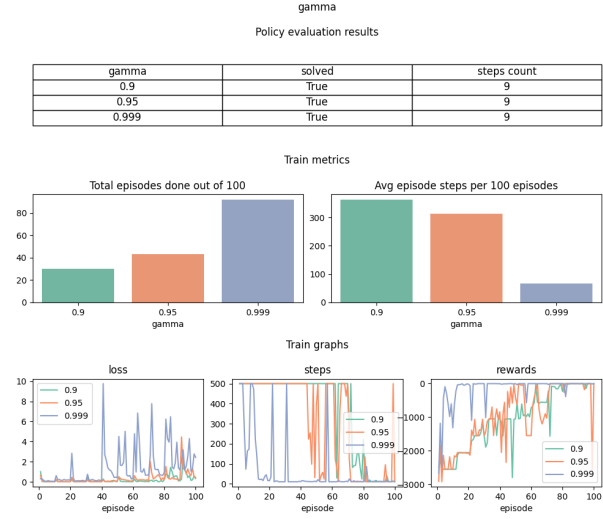


Figure 6: Performance Comparison of Various Learning Rate Values

The results show a clear preference for a high value of Gamma. This value reflects the agent's preference for future rewards over immediate rewards. This is logical, as the most important reward for the agent is the reward it receives for completing the task and ending the game.

3.0.6 Epsilon

The Epsilon balances exploration and exploitation, gradually decreasing to shift from random to more informed actions.

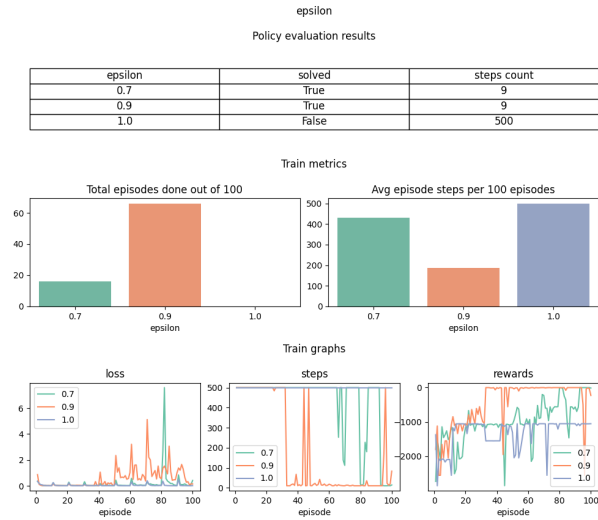


Figure 7: Performance Comparison of Various Learning Rate Values

The results suggest that using an Epsilon value of 0.9 works best. This means it's better to use some exploitation (but not too much) right from the start, instead of exploring as much as possible.

3.0.7 Epsilon Decay

A low Epsilon Decay value quickly reduces exploration, favoring exploitation sooner, while a high value maintains exploration longer for potentially better policy learning.

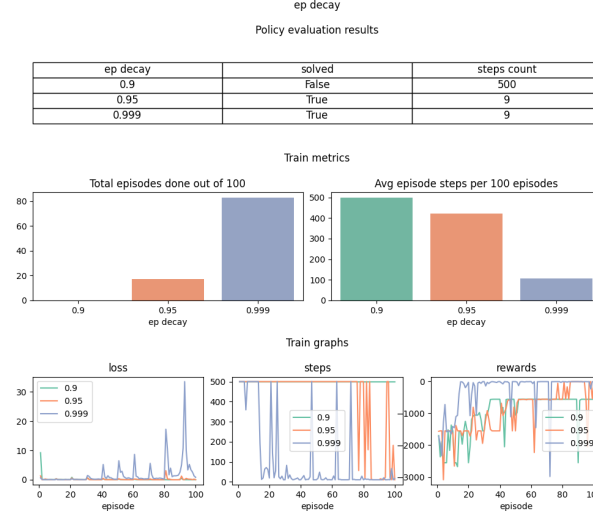


Figure 8: Performance Comparison of Various Learning Rate Values

In the findings, it is evident that a high epsilon decay value is favored. This preference is logical, as the agent requires extensive exploration to achieve the final state.

3.1 Exploring The Alternative Scenarios

Using the hyperparameter values as described in table 1 has proven effective in enabling the agent to find a policy that solves the game in the additional required configurations, such as those involving two boxes.

4 Discussion

The purpose of this project was to apply the Deep Q-Learning algorithm to solve the Sokoban game. During implementation, we encountered two key challenges. First, creating a suitable reward function proved to be challenging, and we had to experiment with various options. Second, selecting the correct hyperparameters, including the discount factor, learning rate, and exploration rate, that played a vital role in the algorithm's performance. Our extensive experimentation led to the identification of hyperparameter values for successfully solving Sokoban. In conclusion, our experiments validate the effectiveness of the Deep Q-Learning algorithm in overcoming the challenges posed by Sokoban.

5 Code

This [Git link](#) contains all the necessary code, along with instructions on running the experiments for each section, and the corresponding Colab notebook.